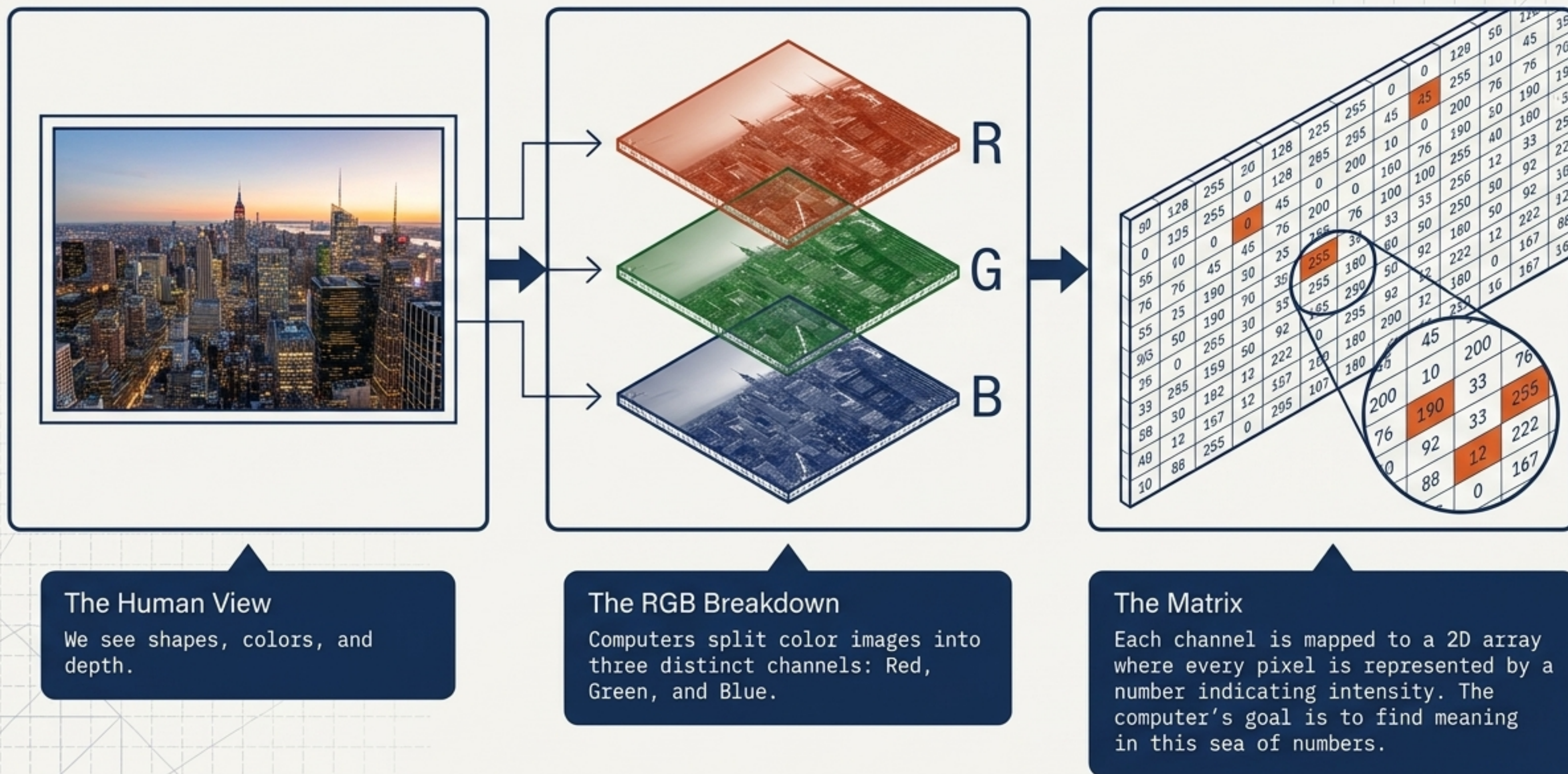




# Demystifying Convolutional Neural Networks

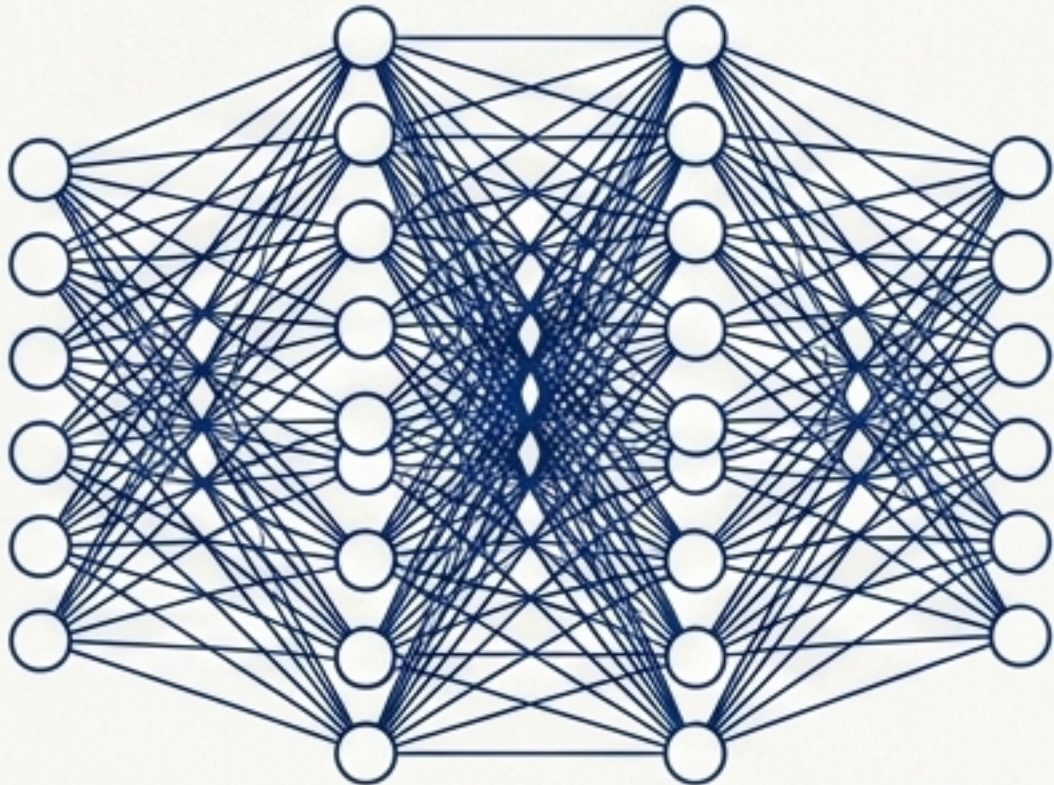
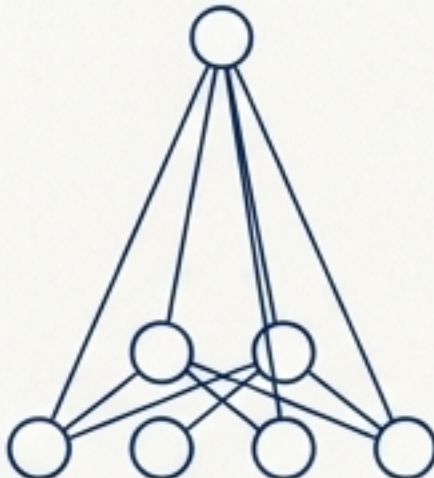
How Computers Learn to 'See'  
the World: A step-by-step guide  
to the architecture behind  
modern image recognition.

# To a computer, an image is just a massive spreadsheet.



# The Parameter Explosion Problem

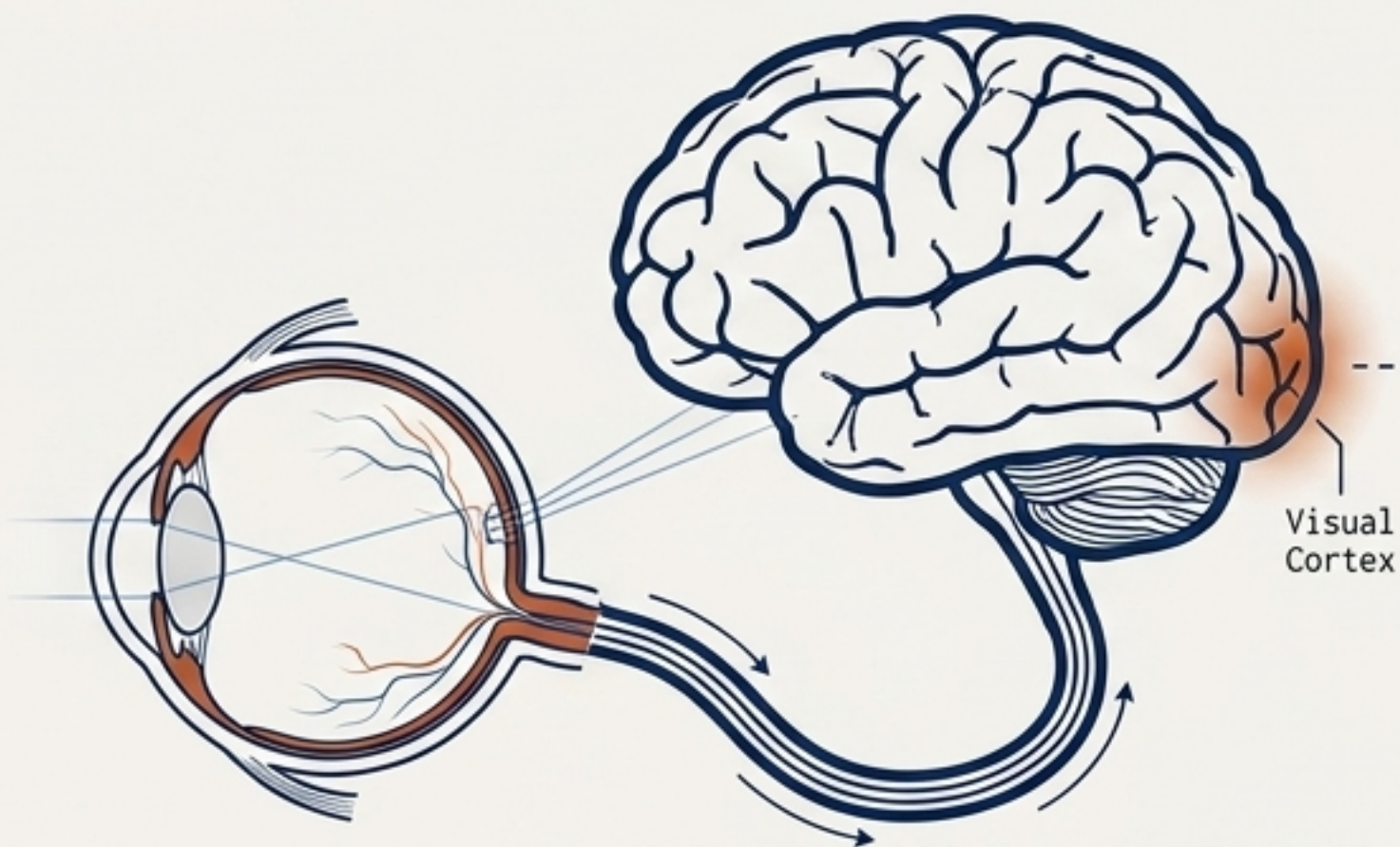
Before CNNs, traditional networks tried to connect every single pixel to every single neuron. For image processing, this approach collapses under its own weight.

| Standard Network (Fully Connected)                                                  |                                                                    | Convolutional Network (CNN)                                                          |                                                                                                      |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
|  |                                                                    |  |                                                                                                      |
| Concept:                                                                            | Connects every single pixel to every single neuron.                | Concept:                                                                             | Connects a neuron to only a small, localized region of pixels.                                       |
| Small Image (28x28):                                                                | 2,352 connections in the very first hidden layer.                  | <u>Verdict:</u>                                                                      | Highly efficient, preserves the physical layout of the image, and requires vastly fewer connections. |
| Normal Image (200x200):                                                             | 120,000 connections instantly.                                     |                                                                                      |                                                                                                      |
| <u>Verdict:</u>                                                                     | Too heavy, too slow, and destroys the spatial layout of the image. |                                                                                      |                                                                                                      |

# Inspired by the Animal Visual Cortex

Convolutional Neural Networks don't look at the entire picture at once. Their architecture is directly inspired by human biology.

## Anatomical Analogy

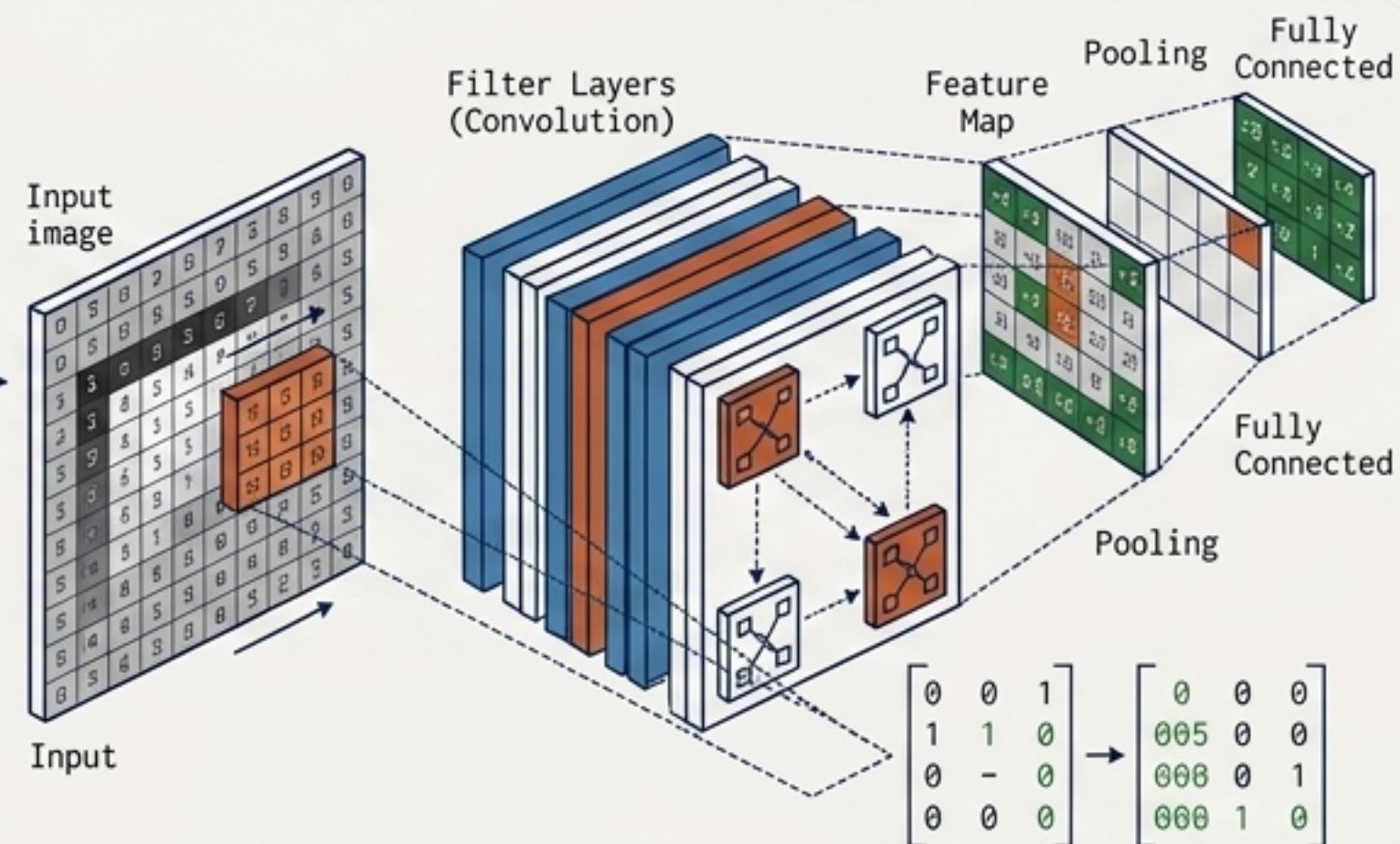


### The Biological Clue

In the animal visual cortex, individual neuronal cells only respond (or "fire") in the presence of highly specific edges. One neuron fires for vertical edges, another for horizontal or diagonal edges.



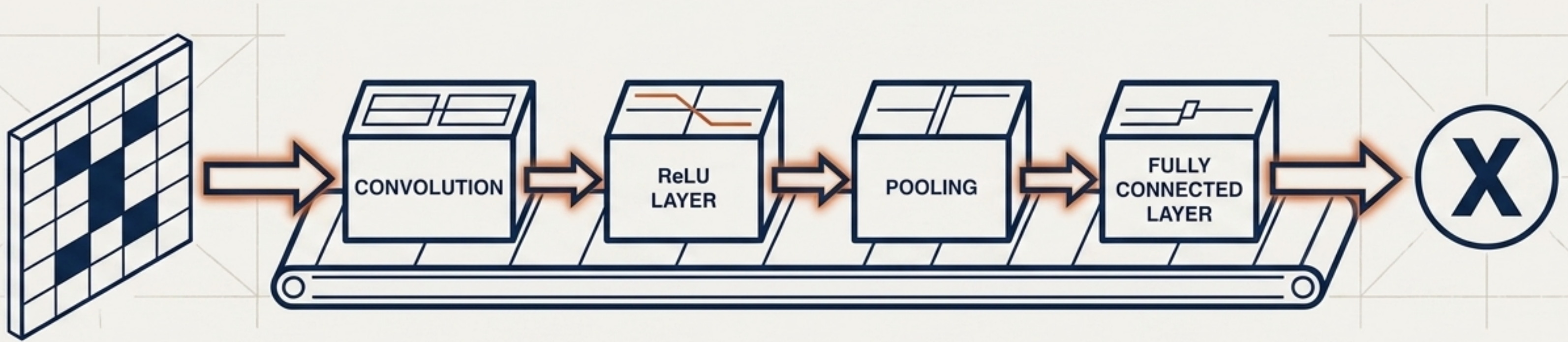
## Digital Translation



### The Digital Translation

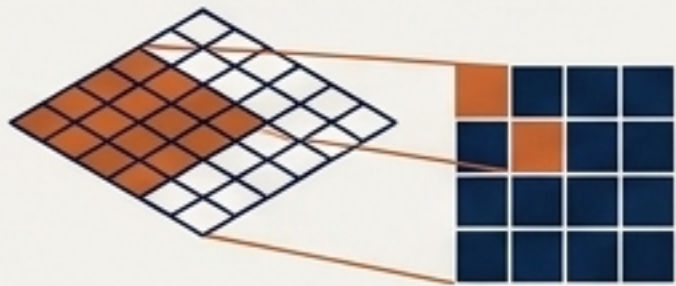
CNNs replicate this. Instead of analyzing the whole image, specialized digital "neurons" sweep across the image, firing only when they detect their designated local feature.

# The Four Pillars of Image Processing



## 1. Convolution

Hunting for localized features using sliding filters.



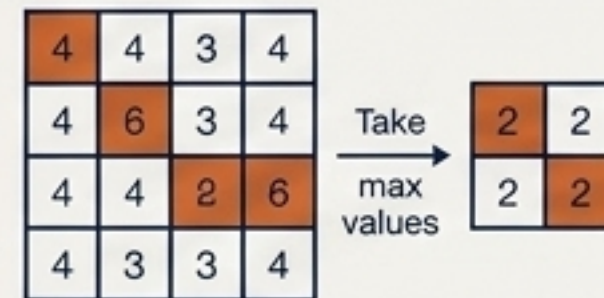
## 2. ReLU Layer

Filtering out the noise and negative values.



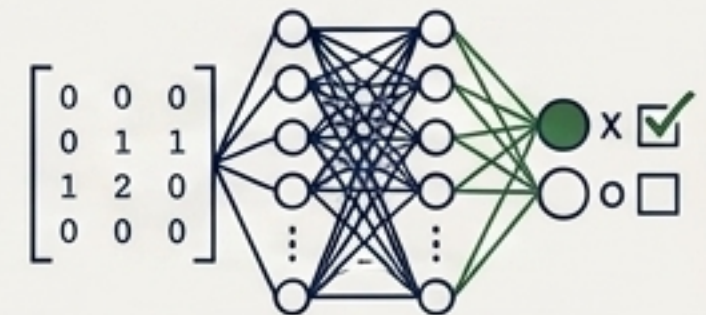
## 3. Pooling

Shrinking the image grid to only the most vital clues.

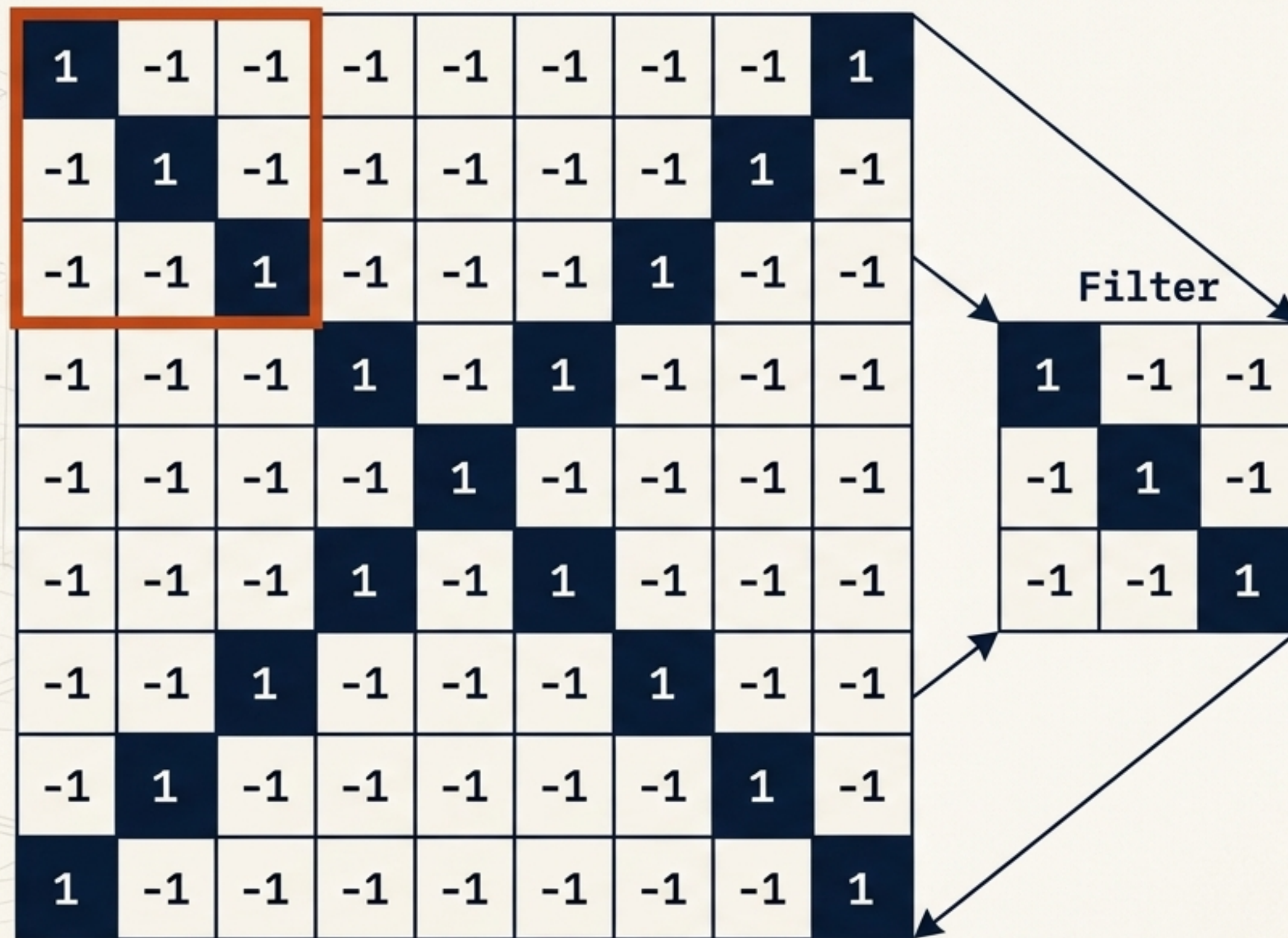


## 4. Fully Connected Layer

Tallying the final score to make a classification.



# Step 1: Convolution — The Matching Game



## Concept:

The network deploys small, specialized templates called **Filters** (or features). It slides this **filter** piece-by-piece across the entire image grid.

## The Translation:

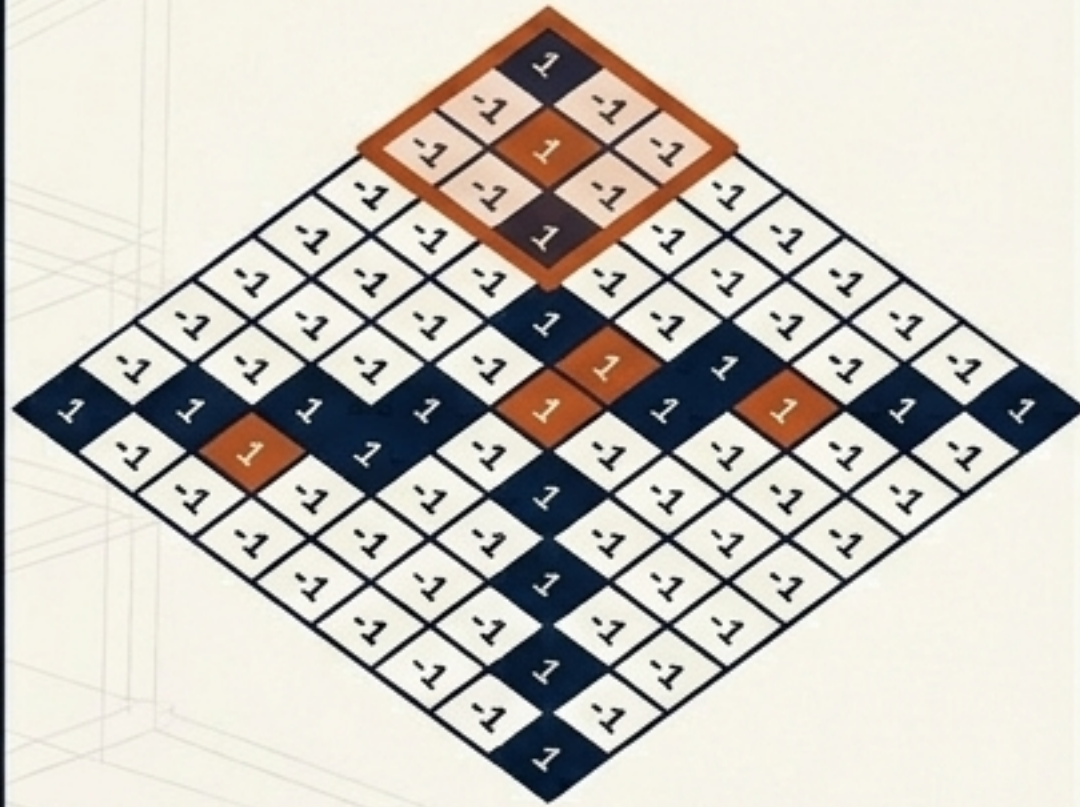
To the computer, **black** pixels have a **value of 1**, and white pixels have a value of **-1**.

## The Goal:

At every stop, it asks: “**Does this tiny 3x3 patch** of the image look exactly like the feature I am searching for?”

# The Math of a Perfect Match

## Step 1: Line Up



Position the feature filter over a section of the image.

## Step 2: Multiply

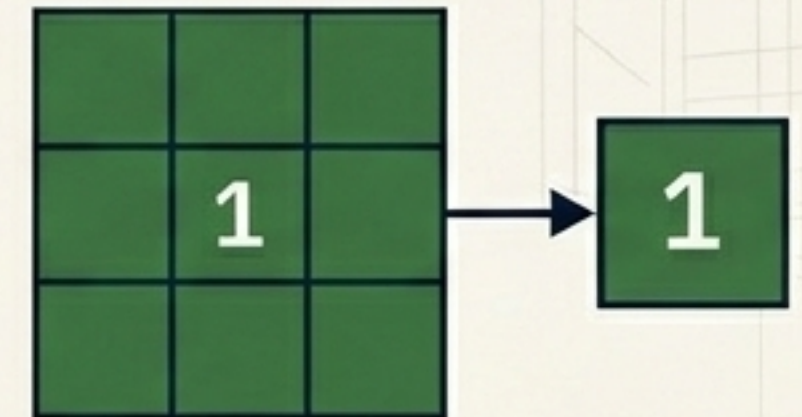
$$\boxed{1} \rightarrow \times \boxed{1} = 1$$

$$\boxed{-1} \rightarrow \times \boxed{-1} = 1$$

Multiply each image pixel by the corresponding feature pixel.  
(Remember: multiplying matching signs yields a positive 1).

## Step 3: Average

$$\frac{(1+1+1+1+1+1+1+1+1)}{9} = 1$$



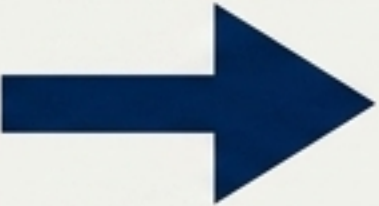
Add the results together and divide by the total number of pixels in the filter.

**The Insight:** When the underlying image perfectly matches the filter, the math results in a perfect score of 1. If they don't match, the score drops.

# Creating the “Heatmap” of Clues

|  |  |  |    |    |    |  |  |
|--|--|--|----|----|----|--|--|
|  |  |  |    |    |    |  |  |
|  |  |  |    |    |    |  |  |
|  |  |  |    |    |    |  |  |
|  |  |  | 1  | -1 | -1 |  |  |
|  |  |  | -1 | 1  | -1 |  |  |
|  |  |  | -1 | -1 | 1  |  |  |
|  |  |  |    |    |    |  |  |
|  |  |  |    |    |    |  |  |
|  |  |  |    |    |    |  |  |

$$\frac{(1+1-1+1+1+1-1+1+1)}{9} = 0.55$$



Feature Map

|       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|
| 0.77  | -0.11 | 0.11  | 0.33  | 0.55  | -0.11 | 0.33  |
| -0.11 | 1.00  | -0.11 | 0.33  | -0.11 | 0.11  | -0.11 |
| 0.11  | -0.11 | 1.00  | -0.33 | 0.11  | -0.11 | 0.55  |
| 0.33  | 0.33  | -0.33 | 0.55  | -0.33 | 0.33  | 0.33  |
| 0.55  | -0.11 | 0.11  | -0.33 | 1.00  | -0.11 | 0.11  |
| -0.11 | 0.11  | -0.11 | 0.33  | -0.11 | 1.00  | -0.11 |
| 0.33  | -0.11 | 0.55  | 0.33  | 0.11  | -0.11 | 0.77  |

## The Output:

After sliding the filter across every possible position, the network generates a brand new matrix called a Feature Map.

## The Heatmap:

This new grid is no longer a picture. It is a spatial heatmap. High values (like 1.00 or 0.77) tell the computer exactly where a specific shape is located in the original image.

## Layering:

The CNN runs dozens of different filters simultaneously, creating a stacked “deck” of these heatmaps for every conceivable shape.

# Step 2: ReLU Layer — Snapping Negatives to Zero

Before

|       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|
| 0.77  | -0.11 | 0.11  | 0.33  | 0.55  | -0.11 | 0.33  |
| -0.11 | 1.00  | -0.11 | 0.33  | -0.11 | 0.11  | -0.11 |
| 0.11  | -0.11 | 1.00  | -0.33 | 0.11  | -0.11 | 0.55  |
| 0.33  | 0.33  | -0.33 | 0.55  | -0.33 | 0.33  | 0.33  |
| 0.55  | -0.11 | 0.11  | -0.33 | 1.00  | -0.11 | 0.11  |
| -0.11 | 0.11  | -0.11 | 0.33  | -0.11 | 1.00  | -0.11 |
| 0.33  | -0.11 | 0.55  | 0.33  | 0.11  | -0.11 | 0.77  |



ReLU Switch

After

|      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|
| 0.77 | 0    | 0.11 | 0.33 | 0.55 | 0    | 0.33 |
| 0    | 1.00 | 0    | 0.33 | 0    | 0.11 | 0    |
| 0.11 | 0    | 1.00 | 0    | 0.11 | 0    | 0.55 |
| 0.33 | 0.33 | 0    | 0.55 | 0    | 0.33 | 0.33 |
| 0.55 | 0    | 0.11 | 0    | 1.00 | 0    | 0.11 |
| 0    | 0.11 | 0    | 0.33 | 0    | 1.00 | 0    |
| 0.33 | 0    | 0.55 | 0.33 | 0.11 | 0    | 0.77 |

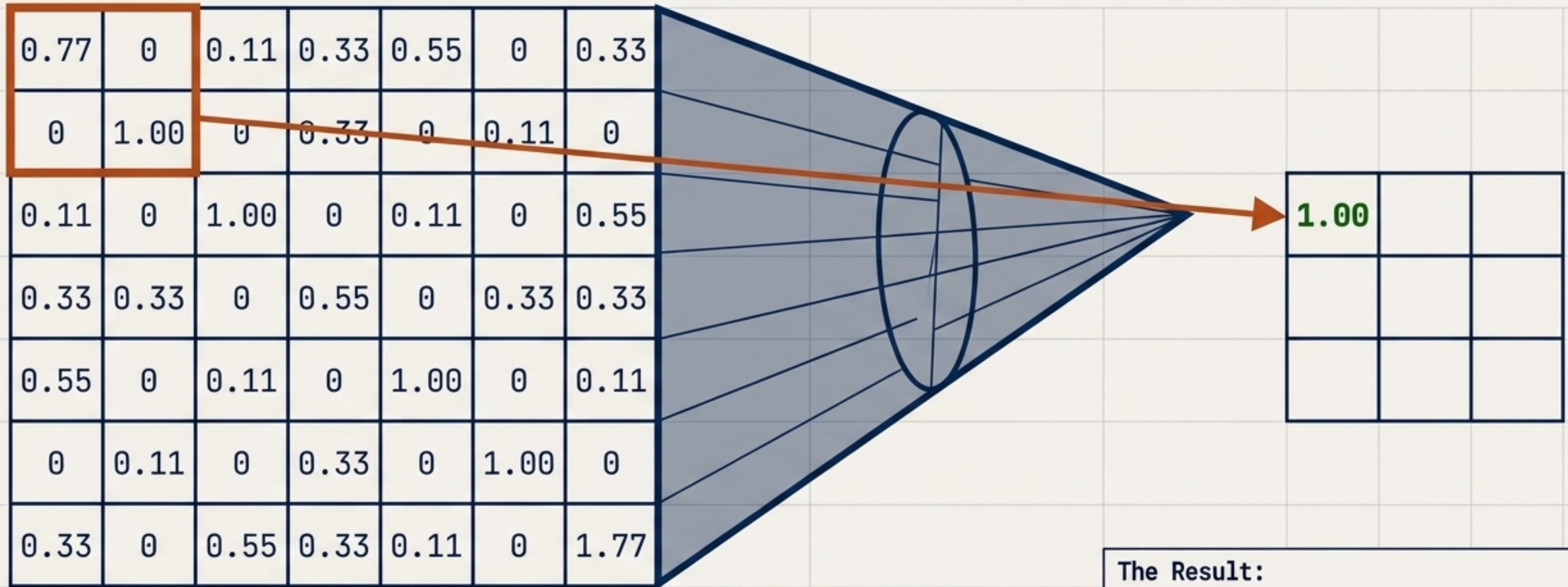
## The Concept:

ReLU (Rectified Linear Unit) acts as a strict mathematical bouncer. It scans the feature map and replaces every negative value with an absolute zero.

## Why Do This?

This crucial step prevents positive and negative values from canceling each other out when summed later. It strictly isolates the exact regions where a feature is present and permanently silences the surrounding noise.

# Step 3: Pooling — Shrinking the Grid, Keeping the Clues



## The Problem:

To keep the network fast, we must compress the massive stack of feature maps.

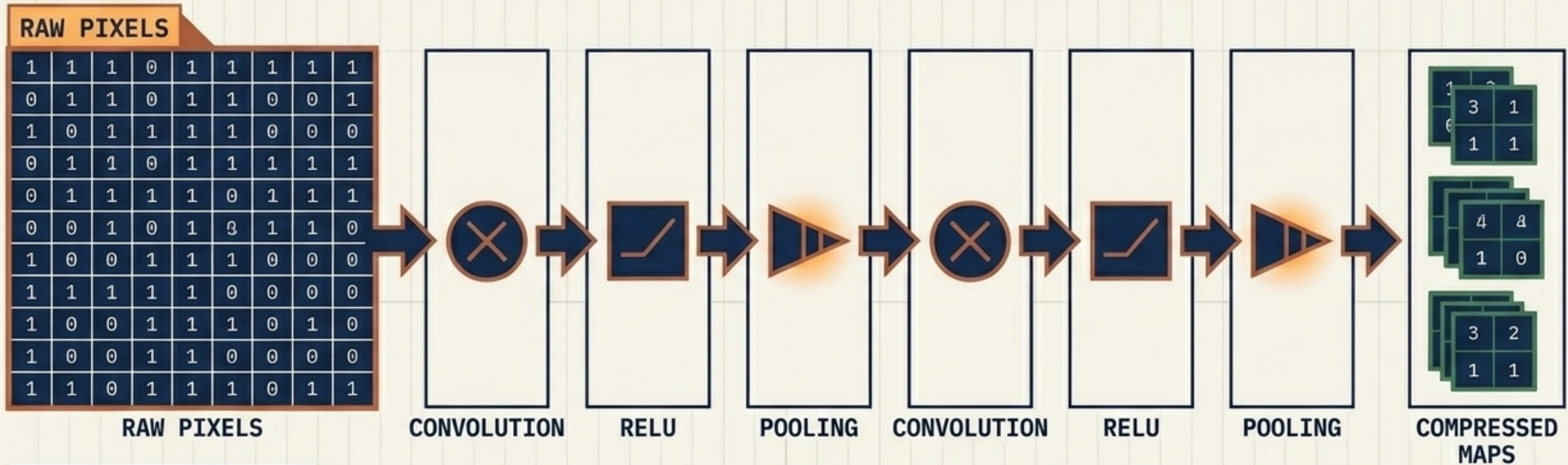
## The Mechanism (Max Pooling):

We slide a 2x2 window (moving 2 strides at a time) across the filtered image. From each block, we extract only the maximum value and discard the rest.

## The Result:

The grid shrinks significantly, but the strongest, most dominant features (the highest numbers) are preserved. The network learns to care less about the exact pixel location, and more about the general presence of a feature.

# Deep Learning: Stacking the Layers



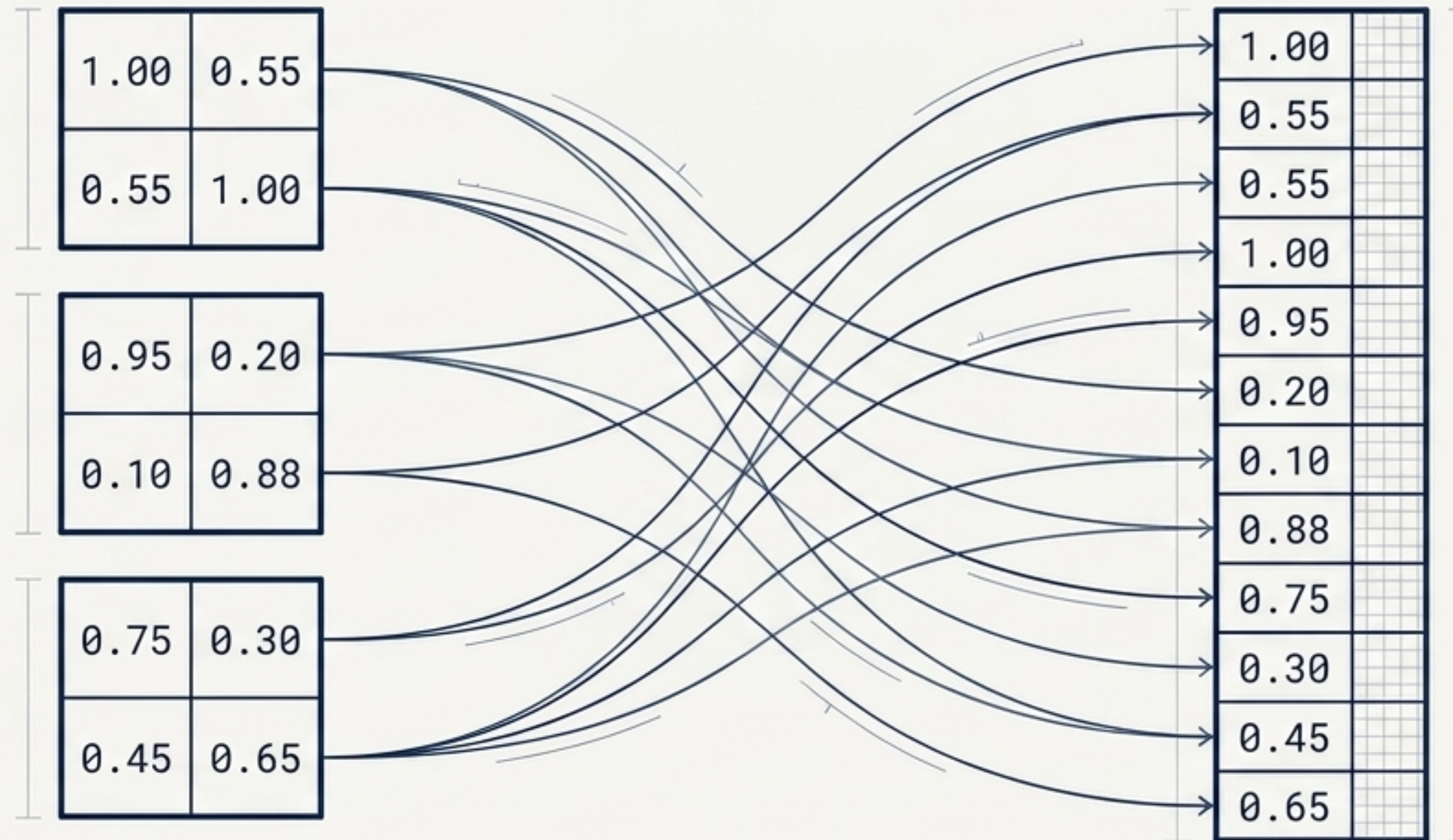
## THE ARCHITECTURE:

A true CNN doesn't execute this process just once. It loops the sequence, feeding the compressed maps into brand new convolution filters.

## BUILDING COMPLEXITY:

1. **EARLY LAYERS:** Detect simple geometry (edges, straight lines).
2. **MIDDLE LAYERS:** Combine lines to find complex shapes (corners, circles).
3. **DEEP LAYERS:** Combine shapes to identify entire objects (eyes, wheels, text characters).

# Step 4: Fully Connected Layer – Preparing the Roster



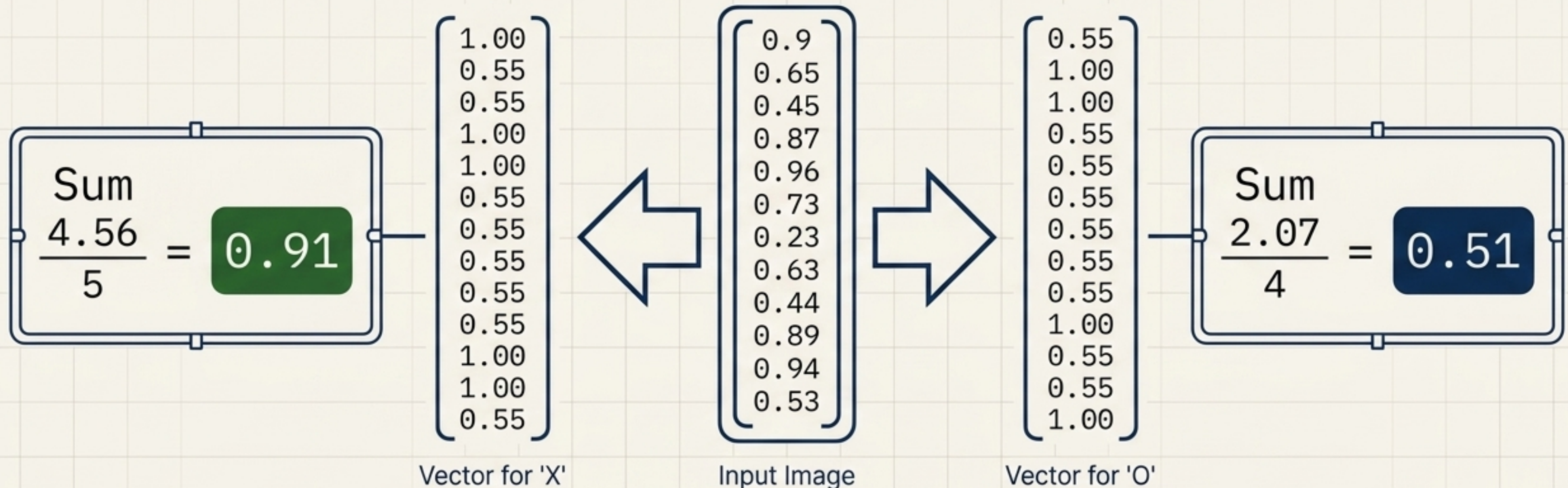
## The Concept:

The 2D structural maps have finished their job. To make a final decision, the network must “flatten” all these highly concentrated 2D clues into a single, 1D vertical list (a vector).

## The Setup:

This single column of numbers represents the ultimate, compressed collection of clues found in the image. It is now ready to be fed into a traditional neural network to make the final classification.

# The Final Verdict: Tallying the Score



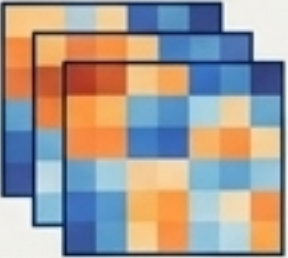
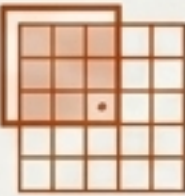
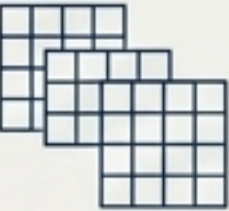
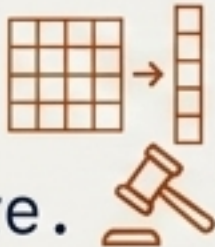
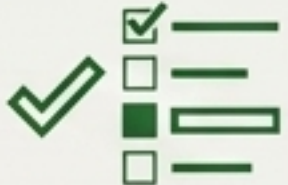
**The Process:** The network compares the final list of clues against stored 'ideal' patterns for every category it has been trained to recognize.

**The Math:** It sums up the overlapping values.



**The Result:** The input heavily matches the expected pattern for an 'X' (0.91) rather than an 'O' (0.51). The CNN confidently outputs its final verdict: The image is an 'X'.

# The Anatomy of Image Recognition: Summary

| Step            | The Metaphor     | Mathematical Action                                                                                                                                                   | Output State                                                                                                                      |
|-----------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Convolution     | Magnifying Glass | <b>Sliding filters</b> to match <b>local pixel features</b> .<br>                  | Feature Maps (Heatmaps).<br>                   |
| ReLU Layer      | The Noise Filter | <b>Snapping</b> all negative values to <b>zero</b> .<br>                           | Rectified Maps.<br>                           |
| Pooling         | The Shrink Ray   | <b>Extracting</b> only <b>maximum values</b> from a <b>moving window</b> .<br>   | Compressed, dominant maps.<br>               |
| Fully Connected | The Judge        | <b>Flattening</b> matrices into vectors and <b>tallying</b> the final score.<br> | <b>Final Categorization Prediction</b> .<br> |